



第4章 串

主讲教师：黄绍辉



在非数值处理、事务处理等问题常涉及到一系列的字符操作。计算机的硬件结构主要是反映数值计算的要求，因此，字符串的处理比具体数值处理复杂。本章讨论串的存储结构及几种基本的处理。



4.1 串类型的定义

串 (字符串)：是零个或多个字符组成的有限序列。
记作： $S = "a_1a_2a_3\ldots"$ ，其中 S 是串名， $a_i (1 \leq i \leq n)$ 可以是字母、数字或其它字符。

串值：双引号括起来的字符序列是串值。

串长：串中所包含的字符个数称为该串的长度。

空串 (空的字符串)：长度为零的串称为空串，它不包含任何字符。

空格串 (空白串)：构成串的所有字符都是空格的串称为空白串。



注意：空串和空白串的不同，例如" "和""分别表示长度为1的空白串和长度为0的空串。

子串 (**substring**)：串中任意个连续字符组成的子序列称为该串的子串，包含子串的串相应地称为主串。特别地，空串是任意串的子串，任意串是其自身的子串。

子串的序号：将子串在主串中首次出现时的该子串的首字符对应主串中的序号，称为子串在主串中的序号（或位置）。

例如，设有串A、B、C分别是：

A="BEIJING"，B="BEI"，C="I"

则B和C都是A的子串，A为主串。B在A的序号为1。C在A中出现了两次，其中首次出现所对应的主串位置是3。因此C在A中的序号为3。

串相等：如果两个串的串值相等(相同)，称这两个串相等。换言之，只有当两个串的长度相等，且各个对应位置的字符都相同时才相等。

通常在程序中使用的串可分为两种：串变量和串常量。

串常量和整常数、实常数一样，在程序中只能被引用但不能不能改变其值，即只能读不能写。通常串常量是由直接量来表示的，例如语句错误(“溢出”)中“溢出”是直接量。

串变量和其它类型的变量一样，其值是可以改变。

串的抽象数据类型定义

ADT String {

数据对象: $D = \{ a_i | a_i \in \text{CharacterSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系: $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

基本操作:

StrAssign(&T, chars)

初始条件: **chars**是一个字符串常量。

操作结果: 生成一个值为**chars**的串**T**。

Concat(&T, S1, S2)

初始条件: 串**S1, S2**已存在。

操作结果: 用**T**返回由**S1**和**S2**联接而成的新串。

StrLength(S)

初始条件：字符串**S**已存在。

操作结果：返回串**S**中的元素个数，称为串长。

SubString(&Sub, S, pos, len)

初始条件：串**S**已存在, $1 \leq \text{pos} \leq \text{StrLength}(s)$ 且 $0 \leq \text{len} \leq \text{StrLength}(s) - \text{pos} + 1$ 。

操作结果：用**Sub**返回串**S**的第**pos**个字符起长度为**len**的子串。

.....

} ADT String

算法4.1，由串的最小操作子集实现定位函数

```
int Index(String S, String T, int pos) {  
    if (pos > 0) {  
        n=StrLength(S); m=StrLength(T); i=pos;  
        while (i<=n-m+1) {  
            SubString(sub, S, i, m);  
            if (StrCompare(sub, T)!=0) ++i;  
            else return i;  
        }//while  
    }//if  
    return 0;  
} //Index
```

4.2 串的存储表示和实现

串是一种特殊的线性表，其存储表示和线性表类似，但又不完全相同。串的存储方式取决于将要对串所进行的操作。串在计算机中有3种表示方式：

- ◆ **定长顺序存储表示**：将串定义成字符数组，利用串名可以直接访问串值。用这种表示方式，串的存储空间在编译时确定，其大小不能改变。
- ◆ **堆分配存储方式**：用一组地址连续的存储单元来依次存储串中的字符序列，但串的存储空间是在程序运行时根据串的实际长度动态分配的。
- ◆ **块链存储方式**：是一种链式存储结构表示。



4.2.1 定长顺序存储表示

这种存储结构又称为串的顺序存储结构。是用一组连续的存储单元来存放串中的字符序列。所谓定长顺序存储结构，是直接使用定长的字符数组来定义，数组的上界预先确定。

定长顺序存储结构定义为：

```
#define MAXSTRLEN 255
```

```
typedef unsigned char SString[MAXSTRLEN+1];
```

此时0下标的值表示串长，即SString[0]等于StrLength(SString)。



1 串的联结操作

```
Status Concat(SString &T, SString S1, SString S2) {  
    if (S1[0]+S2[0]<=MAXSTRLEN) { //未截断  
        T[1..S1[0]]=S1[1..S1[0]];  
        T[S1[0]+1..S1[0]+S2[0]]=S2[1..S2[0]];  
        T[0]=S1[0]+S2[0]; uncut=TRUE;  
    }  
    else if (S1[0] < MAXSTRLEN) { //S2截断  
        T[1..S1[0]]=S1[1..S1[0]];  
        T[S1[0]+1..MAXSTRLEN]=S2[1..MAXSTRLEN-S1[0]];  
        T[0]=MAXSTRLEN; uncut=FALSE;  
    }  
}
```

1 串的联结操作

else { //S1正好填满数组，无法拼接S2

T[0..MAXSTRLEN]=S1[0..MAXSTRLEN];

uncut=FALSE;

}

return uncut;

} //Concat

2 求子串操作

```
Status SubString (SString &Sub, SString S, int pos, int len) {  
    if (pos<1||pos>S[0]||len<0||len>S[0]-pos+1))  
        return ERROR ;  
    Sub[1..len]=S[pos..pos+len-1] ;  
    Sub[0]=len  
    return OK ;  
} //SubString
```

4.2.2 堆分配存储表示

实现方法：系统提供一个空间足够大且地址连续的存储空间(称为**堆**)供串使用。可使用C语言的动态存储分配函数malloc()和free()来管理。

特点是：仍然以一组地址连续的存储空间来存储字符串值，但其所需的存储空间是在程序执行过程中动态分配。

串的堆式存储结构的类型定义

```
typedef struct {  
    char *ch;    // 若非空，按长度分配，否则为NULL  
    int length;  // 串的长度  
} HString ;
```



算法4.4，字符串插入

```
Status StrInsert(HString &S, int pos, HString T) {  
    if (pos<1 || pos>S.length+1) return ERROR;  
    if (T.length) {  
        if (!(S.ch=(char*)realloc(S.ch,(S.length+T.length)*sizeof(char))))  
            exit(OVERFLOW);  
        for(i=S.length-1;i>=pos-1;--i) S.ch[i+T.length]=S.ch[i];  
        S.ch[pos-1..pos+T.length-2]=T.ch[0..T.length-1];  
        S.length+=T.length;  
    }  
    return OK;  
}
```

辅助函数：字符串赋值

```
Status StrAssign(HString &T, char *chars) {  
    if (T.ch) free(T.ch);  
    for(i=0,c=chars; *c; ++i,++c);  
    if (!i) {T.ch=NULL; T.length=0; }  
    else {  
        if (!(T.ch=(char*)malloc(i*sizeof(char))))  
            exit(OVERFLOW);  
        T.ch[0..i-1]=chars[0..i-1];    T.length=i;  
    }  
    return OK;  
} //StrAssign
```

辅助函数：字符串联接

```
Status Concat(HString &T, HString S1, HString S2) {  
    if (T.ch) free(T.ch);  
    if (!(T.ch=(char*)malloc((S1.length+S2.length)*sizeof(char))))  
        exit(OVERFLOW);  
    T.ch[0..S1.length-1]=S1.ch[0..S1.length-1];  
    T.length= S1.length+S2.length;  
    T.ch[S1.length..T.length-1]=S2.ch[0..S2.length-1];  
    return OK;  
} //Concat
```

辅助函数：取子串

```
Status SubString(HString &Sub, HString S, int pos, int len) {  
    if (pos<1 || pos>S.length || len<0 || len>S.length-pos+1)  
        return ERROR;  
    if (Sub.ch) free(Sub.ch);  
    if (!len) { Sub.ch=NULL; Sub.length=0; }  
    else {  
        Sub.ch=(char*)malloc(len*sizeof(char));  
        Sub.ch[0..len-1]=S.ch[pos-1..pos+len-2];  
        Sub.length=len;  
    } return OK;  
} //SubString
```

4.2.3 串的块链存储表示

串的链式存储结构和线性表的串的链式存储结构类似，采用单链表来存储串，结点的构成是：

- ◆ **data域**：存放字符，**data域**可存放的字符个数称为结点的大小；
- ◆ **next域**：存放指向下一结点的指针。

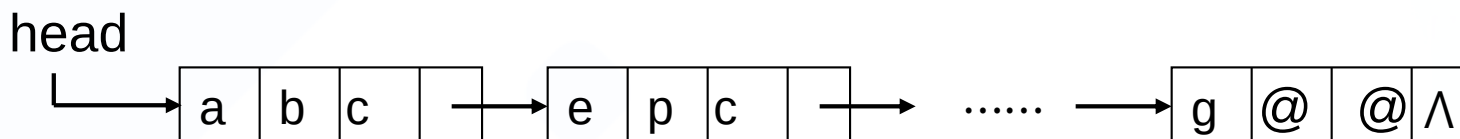
若每个结点仅存放一个字符，则结点的指针域就非常多，造成系统空间浪费，为节省存储空间，考虑串结构的特殊性，使每个结点存放若干个字符，这种结构称为块链结构。



串的块链式存储的类型定义包括：

(1) 块结点的类型定义

```
#define CHUNKSIZE 80  
typedef struct Chunk {  
    char data[CHUNKSIZE] ;  
    struct Chunk *next;  
} Chunk ;
```



CHUNKSIZE=3的块链式存储结构示意图

(2) 块链串的类型定义

```
typedef struct {  
    Chunk *head, *tail;  
    int  curlen ;  
} LString ;
```

在这种存储结构下，结点的分配总是完整的结点为单位，因此，为使一个串能存放在整数个结点中，在串的末尾填上不属于串值的特殊字符，以表示串的终结。

当一个块(结点)内存放多个字符时，往往会使操作过程变得较为复杂，如在串中插入或删除字符操作时通常需要在块间移动字符。在此不作详细讨论。

4.3 串的模式匹配算法

模式匹配（模范匹配）：子串在主串中的定位称为模式匹配或串匹配（字符串匹配）。模式匹配成功是指在主串S中能够找到模式串T，否则，称模式串T在主串S中不存在。

模式匹配的应用在非常广泛。例如，在文本编辑程序中，我们经常要查找某一特定单词在文本中出现的位置。显然，解此问题的有效算法能极大地提高文本编辑程序的响应性能。



4.3.1 Brute-Force模式匹配算法

□ 设S为目标串，T为模式串，且不妨设：

$$S = "s_1s_2s_3...s_n", \quad T = "t_1t_2t_3...t_m"$$

串的匹配实际上是对合法的位置 $1 \leq i \leq n-m+1$ 依次将目标串中的子串 $S[i..i+m-1]$ 和模式串 $T[1..m]$ 进行比较：

◆ 若 $S[i..i+m-1] = T[1..m]$ ：则称从位置 i 开始的匹配成功，亦称模式 T 在目标 S 中出现；

◆ 若 $S[i..i+m-1] \neq T[1..m]$ ：从 i 开始的匹配失败。位置 i 称为位移，当匹配成功时， i 称为有效位移；当匹配失败时， i 称为无效位移。

算法4.5，字符串暴力匹配

```
int Index(SString S, SString T, int pos) {  
    i=pos; j=1;  
    while (i<=S[0] && j<=T[0]) {  
        if (S[i]==T[j]) {++i; ++j;} //继续比较后续字符  
        else { i=i-j+2; j=1; } //i : 回退到前一次比较起点的后一个位置  
                                //j : 回退到第一个字符处  
    }  
    if (j>T[0]) return i-T[0];  
    else return 0;  
}
```

这个算法中，每次匹配失败时，两个下标都需要回退，所以效率比较低。

该算法简单，易于理解。在一些场合的应用里，如文字处理中的文本编辑，其效率较高。

该算法的时间复杂度为 $O(n*m)$ ，其中 n 、 m 分别是主串和模式串的长度。通常情况下，实际运行过程中，该算法的执行时间近似于 $O(n+m)$ 。

改进该算法的关键点

注意到当第一次 $s_i \neq t_j$ 时，其实已经有这个结论了：

$$s_{i-1} = t_{j-1}, \dots, s_{i-j+1} = t_2, s_{i-j} = t_1。$$

换句话说，除了当前位置，之前的子串 $t_1 t_2 \dots t_{j-1}$ 全部匹配成功。那么，如何利用这个结论呢？

4.3.2 模式匹配的一种改进算法

该改进算法是由D.E.Knuth , J.H.Morris和 V.R.Pratt提出来的，简称为KMP算法。其改进在于：

每当一趟匹配过程出现字符不相等时，主串指示器i不用回溯，而是利用已经得到的“部分匹配”结果，将模式串的指示器向右“滑动”尽可能远的一段距离后，继续进行匹配。

例：设有串s="abcbcb", t="abab". 则第一次匹配过程如下。

s="5 a b c b c" i=3

|| | ≠

t="3 a b a" j=3

匹配失败

模式匹配示例



s="5 a b c b c" i=3

|| | ≠

t= "3 a b a" j=3

0 1 2 3 4 5

此时匹配失败，换之前肯定直接让i=2, j=1进行下一次匹配。

- ◆ 在i=3和j=3时，匹配失败。我们考虑第二次匹配的起点：
 - 分析可以发现： $s_2=t_2$ ， $t_2 \neq t_1$ ，必有 $s_2 \neq t_1$ ，所以，i=2的位置无需再次匹配，第二次匹配可以直接从i=3开始，也就是i可以不动；
 - 那么，j要怎么动呢？一定是j=1吗？还真不一定。
- ◆ 直接给结论：匹配失败时，如果充分利用**t串自身的信息**，可以做到**i值不动**，然后j取 ≥ 1 的值，接着进行匹配。
- ◆ 现在的问题是：如何利用t串的信息，计算下次匹配的j值？

不失一般性，设主串 $s="s_1s_2...s_n"$ ，模式串 $t="t_1t_2...t_m"$ 。

当 $s_i \neq t_j (1 \leq i \leq n-m, 1 \leq j < m, m < n)$ 时，主串 s 的指针 i 不必回溯，而模式串 t 的指针 j 回溯到第 $k (k < j)$ 个字符继续比较，则模式串 t 的前 $k-1$ 个字符必须满足(4-2)式，而且不可能存在 $k' > k$ 满足(4-2)式。

$$t_1t_2...t_{k-1} = s_{i-(k-1)} s_{i-(k-2)} \dots s_{i-2} s_{i-1} \quad (4-2)$$

意思是 k 取满足条件的最大值

而已经得到的“部分匹配”的结果为：

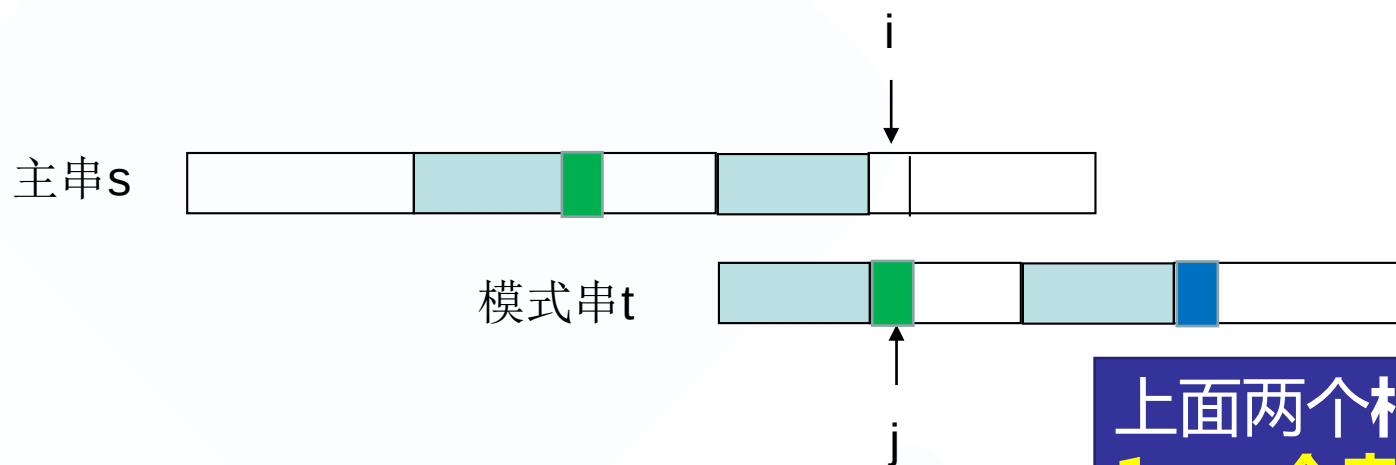
$$t_{j-(k-1)} t_{j-k} \dots t_{j-1} = s_{i-(k-1)} s_{i-(k-2)} \dots s_{i-2} s_{i-1} \quad (4-3)$$

由式(4-1)和式(4-2)得：

$$t_1t_2...t_{k-1} = t_{j-(k-1)} t_{j-k} \dots t_{j-1} \quad (4-4)$$

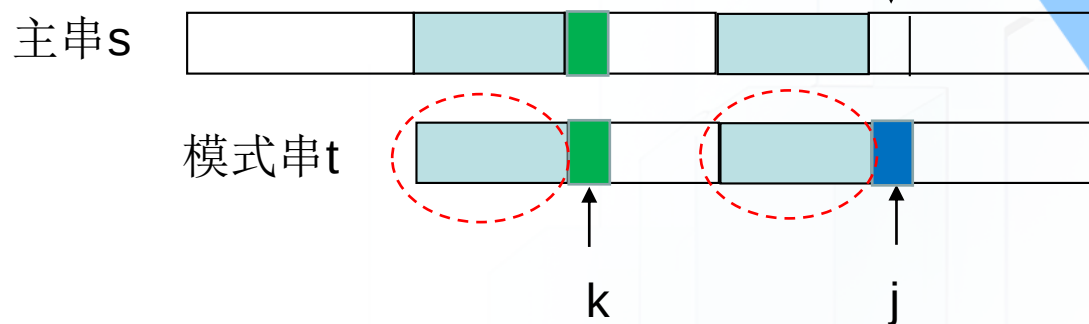
该推导过程可用下图形象描述。实际上，式(4-4)描述了模式串中存在相互重叠的子串的情况。

假设在主串 i ，模式串 j 处失配（如右图）。然后，模式串恰好有两个**相等子串**（图上的虚线圈），则下一次匹配可以直接令 $j=k$ ，然后继续进行匹配（如下图所示）。



KMP算法示例

假设，虚线圈出的两个子串相等



如果把失配时，下一次匹配的位置记为数组 $next$ ，这个例子的 $next[j]=k$ 。

- 上面两个相等子串不能随便选，必须满足：
1. 一个串必须从串首($j=1$)开始；
 2. 另一个串必须以 $j-1$ 的位置为串尾；
 3. 两个串不能完全重叠。
- ✓ k 取值为满足条件1,2,3的最长串的长度。

例题：有模式串"abaabcac"，手工求next数组。
可列出下表求解：

- 1. 一个串必须从串首(j=1)开始；
 - 2. 另一个串必须以j-1的位置为串尾；
 - 3. 两个串不能完全重叠。
- ✓ k取值为满足条件1,2,3的最长串的长度。

模式串的序号j	前j-1个字符	最长相等子串	k值（0起点）	k值（1起点）
1	无定义	无定义	-1（固定值）	0
2	a	空串	0	1
3	ab	空串	0	1
4	aba	a	1	2
5	abaa	a	1	2
6	abaab	ab	2	3
7	abaabc	空串	0	1
8	abaabca	a	1	2

最后一列由倒数第二列加1得到。

可以看到k值就是第三列子串的长度

例题：有模式串"aaaab"，手工求next数组。

可列出下表求解：

模式串的序号j	前j-1个字符	最长相等子串	k值（0起点）	k值（1起点）
1	无定义	无定义	-1（固定值）	0
2	a	空串	0	1
3	aa	a	1	2
4	aaa	aa	2	3
5	aaaa	aaa	3	4

1. 一个串必须从串首(j=1)开始；
2. 另一个串必须以j-1的位置为串尾；
3. 两个串不能完全重叠。
- ✓ k取值为满足条件1,2,3的最长串的长度。

最后一列由倒数第二列加1得到。

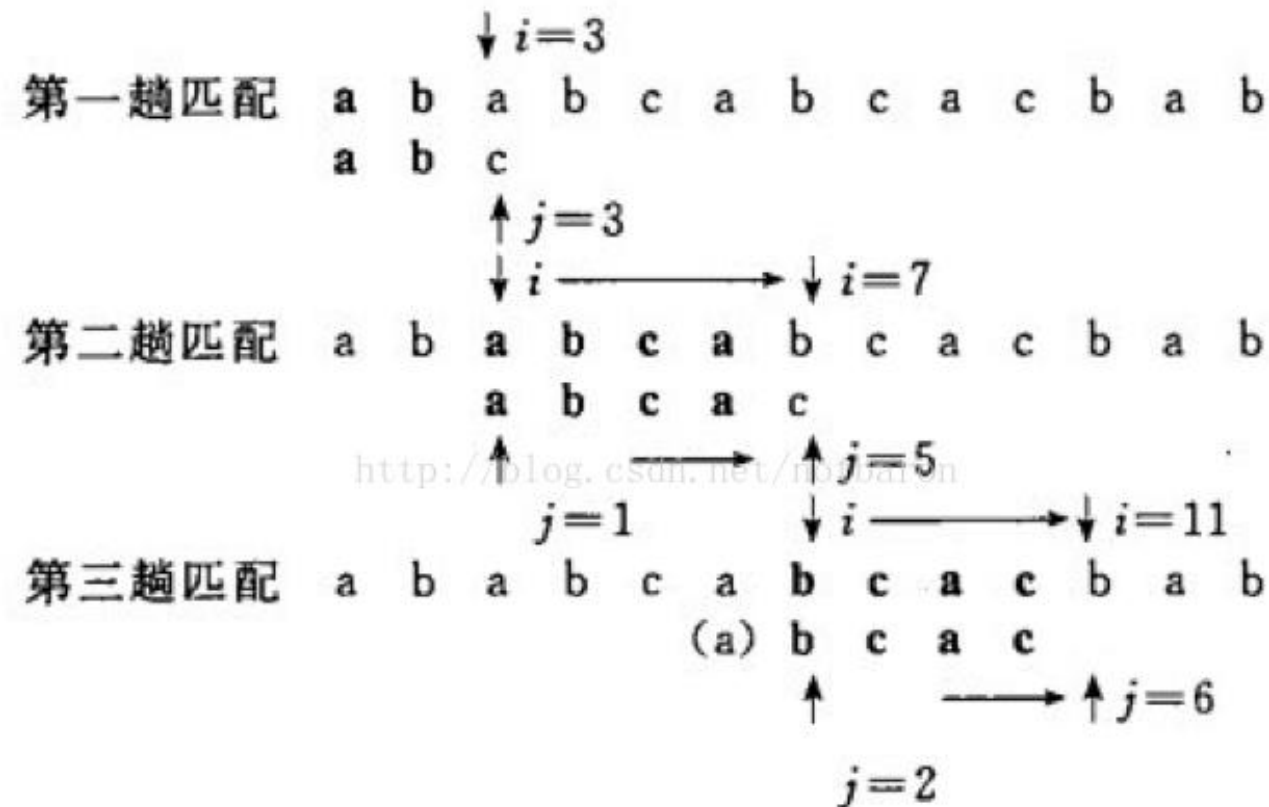
可以看到k值就是第三列子串的长度

next数组直观含义

- ◆ next数组是在失配的时候使用的。
- ◆ 例如模式串 “abaabcac” 对应的next数组， $\text{next}[6]=3$ ，它表示当在 $j=6$ 处失配的时候（红框处），把模式串向右移动，用 $j=3$ 对上失配位置，则前面部分一定是相同的（蓝框处），所以无需再次比较。

S:	a	b	a	a	b	a	b	c	a	b	c	d	d	c	b	a
T:	a	b	a	a	b	c	a	c								
T移动后:				a	b	a	a	b	c	a	c					

有了next数组之后，匹配算法可以更加高效。如图所示：



j	1	2	3	4	5
模式	a	b	c	a	c
next	0	1	1	1	2

✓ 之所以更高效，一是i不再回退，二是j可以根据next的取值跳跃式前进。

图 4.4 改进算法的匹配过程示例

前面给出的是直观解释的next函数，下面是next函数的严格定义：

定义next[j]函数为

$$\text{next}[j] = \begin{cases} 0 & \text{当 } j=1 \text{ 时} \\ \text{Max}\{k | 1 < k < j \text{ 且 } t_1 t_2 \dots t_{k-1} = t_{j-(k-1)} t_{j-k} \dots t_{j-1} \} & \text{该集合不空时} \\ 1 & \text{其它情况} \end{cases} \quad (4-5)$$

上述定义是基于**从1下标**开始存储的字符串，例如课本的SString。

在求得了 $\text{next}[j]$ 值之后，KMP算法的思想是：

设目标串(主串)为 s ，模式串为 t ，并设 i 指针和 j 指针分别指示目标串和模式串中正待比较的字符，设 i 和 j 的初值均为1。若有 $s_i = t_j$ ，则 i 和 j 分别加1。否则， i 不变， j 退回到 $j = \text{next}[j]$ 的位置，再比较 s_i 和 t_j ，若相等，则 i 和 j 分别加1。否则， i 不变， j 再次退回到 $j = \text{next}[j]$ 的位置，依此类推。直到下列两种可能：

(1) j 退回到某个下一个 $\text{next}[j]$ 值时字符比较相等，则指针各自加1继续进行匹配。

(2) 退回到 $j = 0$ ，将 i 和 j 分别加1，即从主串的下一个字符 s_{i+1} ，模式串的 t_1 重新开始匹配。



算法4.6，字符串KMP匹配，红色部分是与暴力匹配不同处

```
int Index_KMP(SString S, SString T, int pos) {
```

```
    i=pos; j=1;
```

```
    while (i<=S[0] && j<=T[0]) {
```

```
        if (j==0 || S[i]==T[j]) {++i; ++j;} //继续比较后续字符
```

```
        else { j=next[j]; } //i : 不动 , j : 回退到next[j]
```

```
    }
```

```
    if (j>T[0]) return i-T[0];
```

```
    else return 0;
```

```
} //Index_KMP
```

如果j一直回退，最终会退到j=0，转入if分支执行。

很显然，Index_KMP函数是在已知next函数值的基础上执行的。现在的问题是：如何求next函数值？

由式(4-5)知，求模式串的next[j]值与主串s无关，只与模式串t本身的构成有关，则可将求next函数值的问题看成是一个模式匹配问题，只是主串和模式串都是t。

由next函数定义可知：

当 $j=1$ 时： $\text{next}[1]=0$ 。这是个固定值。

当 $j>1$ 时，我们来研究一下递推关系：

设 $\text{next}[j]=k$ ，即在模式串中存在：

$t_1 t_2 \dots t_{k-1} = t_{j-(k-1)} t_{j-k} \dots t_{j-1}$ ，其中下标 k 满足 $1 < k < j$ 的某个最大值，此时求 $\text{next}[j+1]$ 的值有两种可能：

(1) 若有 $t_k = t_j$ ：则表明**可以把相等子串再拼一位**，即模式串中有：

$t_1 t_2 \dots t_{k-1} t_k = t_{j-(k-1)} t_{j-k} \dots t_{j-1} t_j$ ，且不可能存在 $k' > k$ 满足上式，即：
 $\text{next}[j+1] = \text{next}[j] + 1 = k + 1$

(2) 若有 $t_k \neq t_j$ ：则表明在模式串中有：

$t_1 t_2 \dots t_{k-1} t_k \neq t_{j-(k-1)} t_{j-k} \dots t_{j-1} t_j$ 。这种情况举例如下：

j	1	2	3	4	5	6	7	8
模式	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	?	?

已经求出 $\text{next}[6]=3$ ，说明： $t_1 t_2 = t_4 t_5 = ab$
想求 $\text{next}[6+1]$ ，发现 $t[3] \neq t[6]$ ，说明无法直接取 $\text{next}[6]+1$ ， $\text{next}[7]$ 只能先尝试能否取 $\text{next}[6]$ ，如果还不行，就需要继续往下取...

- ◆ 如何继续往下取呢？答案是可以将模式向右滑动。向右滑动会自动缩短这个长度。
- ◆ 具体向右滑动多少位合适呢？可以利用之前的结果，将模式向右滑动至模式中next[k]处，和主串中的第j个字符相比较。如下，注意此时j=6，我们正在求解next[6+1]：

j	1 2 3 4 5 6	说明
模式	a b a a b c	
当前位置k=3	a b a a b c	t[3]!=t[6]，尝试3失败
next[3]=1，右滑k=1	a b a a b c	t[1]!=t[6]，尝试1失败
next[1]=0，右滑k=0		不再右滑，执行： ++j; ++k; next[j]=k;



◆ 如前所述，向右滑动的一次之后，还要继续判断：

若 $\text{next}[k]=k'$ ，且 $t_j = t_{k'}$ ，则说明在主串中第 $j+1$ 字符之前存在一个长度为 k' (即 $\text{next}[k]$)的最长子串，与模式串中从第一个字符起长度为 k' 的子串相等。即： $\text{next}[j+1]=k'+1$ ，结束。

同理，若 $t_j \neq t_k$ ，应**将模式继续向右滑动**至将模式中的第 $\text{next}[k']$ 个字符和 t_j 对齐，……，依此类推，直到 t_j 和模式串中的某个字符匹配成功或者不存在任何 $k' (1 < k' < j)$ 满足等式： $t_1 t_2 \dots t_{k-1}$

$$t_{k'} = t_{j-(k'-1)} t_{j-k'} \dots t_{j-1} t_j,$$

则： $\text{next}[j+1]=1$ 。



◆ 小结一下next的求法：（课本p83）

① $\text{next}[1]=0$;

② 设 $\text{next}[i]=j$ ($i \geq 1$)，现在求 $\text{next}[i+1]$ ：

a) 当 $T[i]=T[j]$ 时，说明前缀串和后缀串再加一位依然相等，于是 **$++i; ++j; \text{next}[i]=j$** ;

b) 当 $T[i] \neq T[j]$ 时，说明 $\text{next}[i+1]$ 没那么大，先尝试将j改小： **$j=\text{next}[j]$** ，然后继续判断： $T[i]=T[j]$ ？如果成立，就是a)的情况，于是 **$++i; ++j; \text{next}[i]=j$** ；如果不成立，再将j改小： **$j=\text{next}[j]$** ，然后再判断 $T[i]=T[j]$ ？.....这个过程可能持续到 **$j=0$** ，此时无需再退，然后还是执行 **$++i; ++j; \text{next}[i]=j$** ;

```
if (T[i]==T[j]) //情况a)
    { ++i; ++j; next[i]=j; }
else //情况b)
    {
        while (T[i]!=T[j] && j!=0)
        {
            j=next[j]; //尝试改小j
        }
        ++i; ++j; next[i]=j;
    }
```

去掉a)

```
while (T[i]!=T[j] && j!=0)
{
    j=next[j];
}
++i; ++j; next[i]=j;
```



- ◆ 上述的代码是求某个next[i]，如果是计算整个next数组，需要套一层循环，如下左侧代码：

```
i=1; next[1]=0; j=0;
while (i<T[0])
{
    if (j!=0 && T[i]!=T[j])
        j=next[j];
    else
        { ++i; ++j; next[i]=j; }
}
```

如果把两个分支条件对换一下，
就是右边的代码：

```
i=1; next[1]=0; j=0;
while (i<T[0])
{
    if (j==0 || T[i]==T[j])
    {
        ++i; ++j; next[i]=j;
    }
    else
        j=next[j];
}
```



算法4.7, next函数

```
void get_next(SString T, int next[]) {  
    i=1; next[1]=0; j=0;  
    while (i<T[0]) {  
        if (j==0 || T[i]==T[j]) {  
            ++i; ++j; next[i]=j; //无法滑动或无需滑动，可以赋值  
        }  
        else j=next[j]; //向右滑动next[j]  
    }  
} //get_next
```

对于右侧的模式，在 $j=4$ 失配的情况下，也即 $T[4] \neq S[i]$ ，此时按照next值，还需尝试：

- ① $T[3]$ 是否等于 $S[i]$ ？
- ② $T[2]$ 是否等于 $S[i]$ ？
- ③ $T[1]$ 是否等于 $S[i]$ ？

实际上，因为 $T[1] \sim T[4]$ 均相等，可以直接一步到位比较 $T[1]$ 是否等于 $S[i+1]$ 。也就是说，上述 $T[3]$ 、 $T[2]$ 比较可以避免。

j	1 2 3 4 5
模式	a a a a b
next[j]	0 1 2 3 4
nextval[j]	0 0 0 0 4

由此，可以得到next的升级版nextval。也就是先按照之前next的方法，计算nextval[i]，如果发现 $T[i] = T[j]$ ，那么nextval[i]的值需要修正为之前nextval[j]的值。

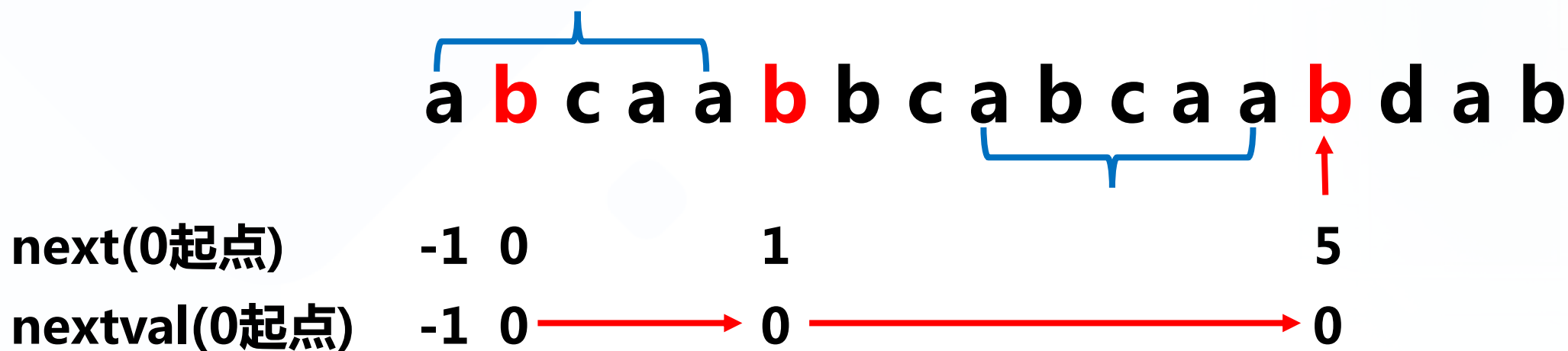


算法4.8, nextval函数

```
void get_nextval(SString T, int nextval[]) {  
    i=1; nextval[1]=0; j=0;  
    while (i<T[0]) {  
        if (j==0 || T[i]==T[j]) {  
            ++i; ++j;  
            if (T[i]!=T[j]) nextval[i]=j; //取新值, 这也是next函数的写法  
            else nextval[i]=nextval[j]; //修正为 j 对应的值  
        }  
        else j=nextval[j]; //向右滑动nextval[j], 这个和next函数一致  
    }  
} //get_nextval
```

nextval可以手工求吗？答案是可以，求法与next类似。熟练之后甚至不用列表，直接手搓就行。例：有模式串"abcaabbcbcabcaabdab"，手工求next和nextval数组。

- ①前两个值固定填-1和0；
- ②都是从左往右填，假设填到下图的b处，则查看b之前的子串，直接寻找两个最长相等子串；
- ③nextval在next的基础上，比较当前指向的字母与next指向的字母，若相等则修正；
- ④1起点则在0起点基础上加一。



4.4 串操作应用举例

4.4.1 文本编辑

- ◆ 本文编辑程序是一个面向用户的系统服务程序，广泛用于源程序的输入和修改，甚至用于报刊和书籍的编辑排版以及办公室的公文书信的起草和润色。
- ◆ 一般包括串的查找、插入和删除等基本操作。
- ◆ 对用户来讲，一个文本（文件）可以包括若干页，每页包括若干行，每行包括若干文字。
- ◆ 对文本编辑程序来讲，可把整个文本看成一个长字符串，称文本串，页是文本串的子串，行又是页的子串。为简化程序复杂程度，可简单地把文本分成若干行。



例：下面的一段源程序可以看成一个文本串。

```
main(){
float a,b,max;
scanf("%f,%f",&a,&b);
if (a>b) max=a;
else max=b;
};
```

- ◆ 在编辑时，为指示当前编辑位置，程序中要设立页指针、行指针、字符指针，分别指示当前页，当前行，当前字符。因此程序中要设立页表、行表便于查找。参考课本图4.8。

[illegible]

4.4.2 建立词索引表

- ◆ 信息检索是计算机应用的重要领域之一。
- ◆ 举个例子图书馆书名检索系统。
一般来讲通过书名关键词检索读者感兴趣的书是很重要的。因为读者一般不知道自己感兴趣的书的全名，可能会凭印象提供一些关键词。
- ◆ 检索系统就要求能够根据读者提供的关键词，显示所有含有该关键词的书目。

书号	书名
005	Computer Data Structure
010	In troduction to Data Structure
023	Fun dame ntal of Data Structure
034	The Desig n and An alysis of Computer Algorithms
050	In troduct ion to Numerical An alysis
067	Numerical An alysis

关键词	书号索引
algorithms	034,
an alysis	034,050,067,
computer	005,034,
data desig n	005,010,023,
fun dame	034,
ntal in	023,
troductio n	010,050,
nu merical	050,067,
structure	005,010,023,



操作设计

1. 从书名文件中读取一个书名串；
2. 从书名串中提取所有关键词到词表中；
3. 对词表中的每一个关键词，在关键词索引表中进行查找并作相应的插入操作；
4. 一个判断是否为关键词的机制，创建一个常用词表（**an**、**a**、**of**、**the**），如果从书名提取出的词不与常用词表中任意词相等，即为关键词；
5. 在索引表中查询关键词可能出现以下情况：
 - 1) 索引表中已有此关键词的索引项，只要在该项中插入书号索引即可；
 - 2) 需在索引表中插入词关键词的索引项，插入应按照词典有序原则进行；

数据结构设计

1. 词表是一个线性表，因为一本书名只有若干个关键词，其数量有限，则采用顺序存储结构即可。每个词都是一个字符串。
2. 索引表是一个有序表，在生成过程中需要频繁进行插入操作；索引表也为了查找用，为了提高查找效率，应采用顺序存储结构；
3. 索引表中每个索引项包含两个内容：其一是关键词；其二是书号索引；
4. 关键词：索引表是常驻机构，采用堆分配存储表示的串类型；
5. 书号索引：书号索引在索引表的生成过程中逐个插入，且不同关键词的书号索引个数不等，甚至可能相差很多，宜采用链表结构的线性表；

代码设计

```
#define MaxBookNum 1000    //假设只对1000本书建立索引表
#define MaxKeyNum 2500     //索引表的最大长度
#define MaxLineLen 500    //书名串最大长度
#define MaxWordNum 10     //词表最大容量
```

```
typedef struct{
    char *item[];    //字符串的数组
    int last;        //词表长度
}WordListType;    //词表类型
```

```
typedef struct{
    HString key;    //关键词
    LinkList bnolist; //存放书号索引的链表
}IdxTermType;    //定义索引项

typedef int ElemType;    //定义链表的数据元素类型为整型

typedef struct{
    IdxTermType item[MaxKeyNum+1];
    int last;
}IdxListType    //索引表类型

char * buf;    //书名串缓冲区
WordListType wdlist;    //词表
```

算法4.9 主函数

```
void main() {  
    if(f=fopen("BookInfo.txt","r"))  
        if(g=fopen("BookIdx.txt","w")) {  
            InitIdxList(idxlist); //初始化索引表idxlist为空表  
            while(!feof(f)) {  
                GetLine(f); //从文件f中读入一个书目信息到buf  
                ExtractKeyWord(BookNo); //从buf提取关键词到词表，书号存入BookNo中  
                InsIdxList(idxlist, BookNo); //将书号为BookNo的关键词插入索引表  
            }  
            PutText(g, idxlist); //将生成的索引表idxlist输出到文件g  
        }  
}
```

算法4.10 索引表插入算法、算法4.11 获取第i个关键词函数

```
Status InsIdxList(IdxListType &idxlist, int bno){  
    for(i=0; i<wdlist.list; ++i){  
        GetWord(i,wd);  
        j=Locate(idxlist,wd,b);  
        if(!b) InsertNewKey(idxlist, j, wd);    //插入新的索引项  
        if(!InsertBook(idxlist,j,bno)) return OVERFLOW;    //插入书号索引  
    }  
    return OK;  
}  
  
void GetWord(int i, HString &wd){  
    p=*(wdlist.item+i);    //取词表中第i个字符串  
    StrAssign(wd,p);        //生成关键词字符串  
}
```

```

int Locate(IdxDListType &idxlist, HString wd, Boolean &b){
    for(i=idxlist.last-1; m=StrCompare(idxlist.item[i].key,wd)>0; --i);
    if(m==0) {b=TRUE; return i;}    //找到
    else{b=FALSE; return i+1;}
}

void InserNewKey(int i, StrType wd){
    for(j=idxlist.last-1;j>=i;--j)    //后移索引项
        idxlist.item[j+1]=idxlist.item[j];
    StrCopy(idxlist.item[i],wd);    //串赋值
    InitIdxList(idxlist.item[i].bnolist);    //初始化书号索引表为空表
    ++idxlist.last;
}

```

- ✓ 算法4.12 查询关键词是否存在
- ✓ 算法4.13 插入新关键词
- ✓ 算法4.14 插入索引

```

Status InsertBook(IdxDListType &idxlist, int i, int bno){
    if(!MakeNode(p,bno)) return ERROR;    //分配失败
    Appand(idxlist.item[i].bnolist, p);    //插入新的书号索引
    return OK;
}

```

结束

谢谢

